

Swiss Joint Master in Computer Science

13081/43081 Systems Verification

Beyond Shared Memory in Promela

Prof. Ulrich Ultes-Nitsche

Department of Informatics, University of Fribourg

Process communication

How have processes communicated up to now?

- ▶ via shared variables
 - ▶ that is a model that represents shared-memory communication in real systems
 - ▶ that is a realistic model as long as all processes run on shared hardware
- ▶ What if we want to model systems that are distributed and communicate via a network?
 - ▶ Promela uses the concept of *channels*

Communication models

there are basically two different types of communication: *synchronous* and *asynchronous*

- ▶ synchronous communication:
 - ▶ only takes place when sender and receiver at the same time are ready to communicate
 - ▶ (non-computer-science) example: telephone

- ▶ asynchronous communication:
 - ▶ also takes place when sender and receiver at the same time are not ready to communicate
 - ▶ (non-computer-science) example: letter

Synchronous channels in Promela

channels are declared using

```
chan Name = [ size ] of { data_type };
```

- ▶ size determines how many data units the channel can store
 - ▶ if size is 0, then the channel cannot store anything
 - ▶ such a channel is a synchronous channel

communication in synchronous channels:

- ▶ if a sender wants to send, but the receiver is not ready to receive, the send operation is blocked until the receiver gets ready
- ▶ if a receiver wants to receive, but the sender is not ready to send, the receive operation is blocked until the sender gets ready

Writing to and reading from channels

if a channel was declared by

```
chan name = [ size ] of { data_type };
```

then

```
name! data ;
```

writes data in the channel

```
name? data ;
```

reads data from the channel

An example to “play with”

```
chan global = [0] of {bool};

proctype write() {
do
:: skip;
:: global!0;
od;}

proctype read() {
do
:: skip;
:: global?0;
od;}

init{atomic{run write(); run read();}}}
```

we play around with **Channel.pml**

Uninstantiated channels

```
chan name ;
```

creates an uninstantiated channel variable (to which a channel can be assigned later)

Channel operations

`name! data`

writes data into channel name

`name? data`

reads data from channel name

`name? [ data ]`

evaluates to true, if a corresponding read operation could be performed

`name?< data >`

reads data from channel name without removing it

Channel operations

```
name !! data
```

writes data into channel name in an ordered way

```
name ?? data
```

reads the oldest matching data from channel name

```
name ?? [ data ]
```

evaluates to true, if a corresponding read operation could be performed

```
name ?? <data>
```

reads the oldest matching data from channel name without removing it

Channel operations

empty(name)

checks whether or not channel name is empty

nempty(name)

checks whether or not channel name is not empty

full(name)

checks whether or not channel name is full

nfull(name)

checks whether or not channel name is not full

len(name)

returns the number of messages currently stored in channel name

Variable evaluation

```
eval( variable )
```

turns a variable's value into a constant

```
channel name = [0] of { byte };  
byte var = 42;  
... name?var; ...
```

the read operation will always be executable (if there is a write operation to the channel)

```
channel name = [0] of { byte };  
byte var = 42;  
... name?eval( var ); ...
```

only when 42 is written to the channel will the read operation be executable

Mutex for more than Two Processes

Lesley Lamport's *Bakery Algorithm* (like in Swiss post offices or railway stations).

- ▶ a process that wants to enter the critical section picks a ticket with a number
 - ▶ requires a *ticket issuer*
- ▶ a *scheduler* communicates which ticket number is next to enter the critical section
 - ▶ requires feedback when a process leaves the critical section

The Bakery Algorithm

```
1  #define numProcesses 3
2
3  bool start;
4  byte procID[numProcesses];
5
6  chan ticket = [0] of {byte};
7  chan go = [0] of {byte};
8
9  proctype issuer() {
10     byte count = 0;
11     do
12         :: ticket!count; count = (count + 1) % numProcesses;
13     od
14 }
15
16 proctype scheduler() {
17     byte count = 0;
18     do
19         :: go!count;
20         go?eval(count);
21         count = (count + 1) % numProcesses;
22     od;
23 }
24
25 proctype MutExBakery() {
26     byte t;
27     start;
28     do
29         :: ticket?t;
30         go?eval(t);
31         critical: /* critical section */
32         go!t;
33     od;
34 }
35
36 init {
37     run issuer(); run scheduler();
38     byte k = numProcesses;
39     do
40         :: procID[k-1] = run MutExBakery();
41         k--;
42         if
43             :: k<1; break;
44             :: else;
45         fi
46     od;
47     start = true;
48 }
```

byte i, j ;

proc "random" :

select a pair of
two different proc. IDs.

do :: $i = (i+1) \% \text{numProcesses};$

:: break;

od

do

:: $j = (j+1) \% \text{numProcesses}$

:: $i \neq j$; break;

od

How can we verify the Bakery Algorithm?

MutExBakeryVerification.pml

A Simplified Version

MutExBakeryVerification2.pml

A crypto protocol: the Needham-Schroeder protocol

Bob

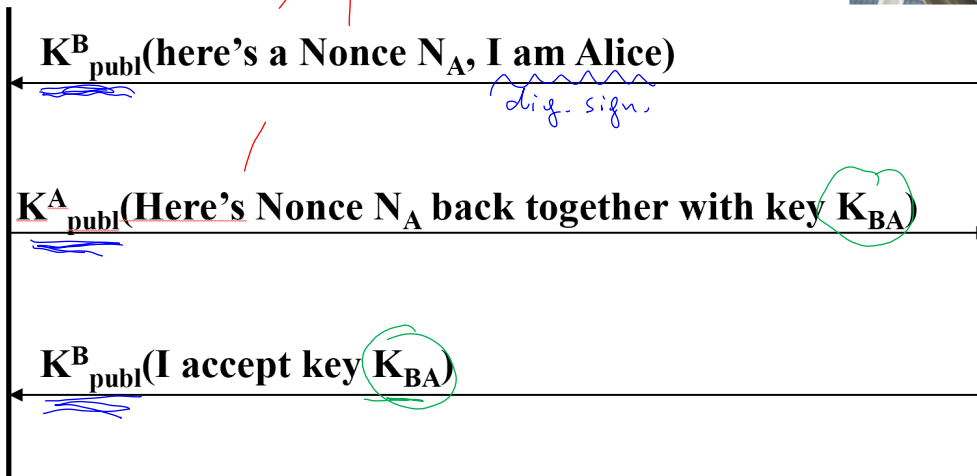


Alice



Not more than once
(big random number)

public key
crypto



K_{BA} : session key
(random num.) for Alice Bob

How do we model perfect cryptography?

With names used in a synchronous channel!?!

A Promela Model

```
1      mtype = {PubA, PubB, PubM, Nonce, Key, A, B, M, void};
2      mtype target;
3
4      #define PriA PubA
5      #define PriB PubB
6      #define PriM PubM
7      chan global = [0] of {mtype, mtype, mtype};
8
9      active proctype Alice(){
10         mtype key;
11         global!PubB, Nonce, A;
12         global?PriA, Nonce, key;
13         global!PubB, key, void;
14     }
15
16     active proctype Bob(){
17         mtype nonce;
18         mtype identity;
19         global?PriB, nonce, identity;
20         identity==A;
21         global!PubA, nonce, Key;
22         global?PriB, Key, void;
23     }
```

NeedhamSchroeder1.pml

Needham-Schroeder with 2 Recipients

```
...
9      active proctype Alice(){
10      mtype key;
11      if
12          :: target=B;
13              global!PubB,Nonce,A;
14              global?PriA,Nonce,key;
15              global!PubB,key,void;
16          :: target=M;
17              global!PubM,Nonce,A;
18              global?PriA,Nonce,key;
19              global!PubM,key,void;
20      fi;
21      }
...
```

NeedhamSchroeder2.pml

Analysing Crypto-Protocols

Dolev-Yao attacker model:

- ▶ They can intercept any sent message.
- ▶ They are eligible users and can send messages to everyone.
- ▶ They receive messages of all users.
- ▶ They can send messages to other users using false identities.
- ▶ They possess the content of all previously sent encrypted messages to which they possess a matching key.
- ▶ They possess all previously sent messages.
- ▶ They can encrypt all cleartext messages with any key they possess.

Needham-Schroeder Dolev-Yao Attacker

```
...
23   active proctype MitM(){
24   mtype part1 , part2;
25   do
26       :: global?PriM , part1 , part2;
27       :: do
28           :: global!PubA , part1 , part2;
29           :: global!PubA , part2 , part1;
30           :: global!PubA , part1 , void;
31           :: global!PubA , part2 , void;
32           :: global!PubA , void , part1;
33           :: global!PubA , void , part2;
34           :: global!PubA , void , void;
35           :: global!PubB , part1 , part2;
36           :: global!PubB , part2 , part1;
37           :: global!PubB , part1 , void;
38           :: global!PubB , part2 , void;
39           :: global!PubB , void , part1;
40           :: global!PubB , void , part2;
41           :: global!PubB , void , void;
42       :: break;
43   od;
44   od;
45   end:
46   }
```

NeedhamSchroederAttack.pml

The Attack against the Needham-Schroeder Protocol

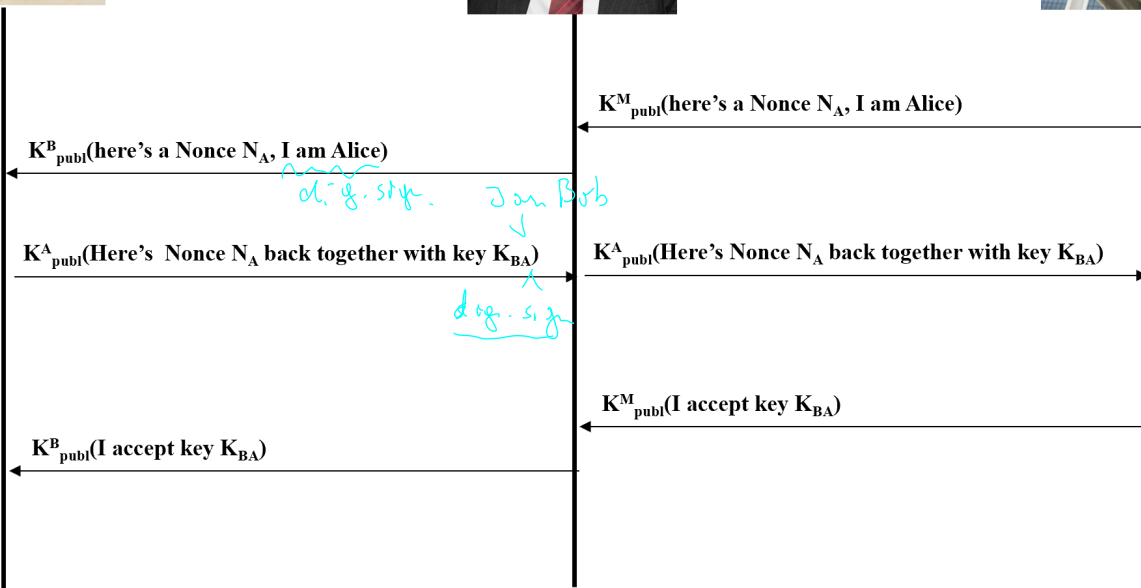
Bob



„man in the middle“



Alice



That's all

Do you have questions?